Computer exercise for WI4201
Iterative Solvers and Domain Decomposition for the Poisson Equation

Instructions and assessment criteria

- **Assessment.** The assessment weights are 40 points for the pen-and-paper questions and 60 points for implementation and numerical analysis. The overall take-home exam grade must be at least 5.0 to pass the course.
- **Submission.** Submit your group's PDF report and source code through Brightspace by the deadline stated there. Submissions by email will not be accepted.
- **Group and report.** Work in groups of **three**. Put all group members' names and the group number on the front page. The report has a **maximum length of 8 pages**, excluding the title page and references. Write it in LaTeX, Word, or another suitable typesetting system, and answer the questions in order.
- **Implementation.** Use Julia, Python, or MATLAB. Submit the source code as separate files, with instructions for running the experiments and a list of required software packages. The documented commands should reproduce the results without interactive input. State which routines you implemented yourself and which library routines you used.
- **Explanations.** Justify your answers and implementation choices; code comments may support these explanations. Interpret the plots and tables rather than presenting numerical results alone.
- **Oral examination.** There is one overall grade for the take-home exam, determined directly in the oral examination. Your report and code provide the basis for this examination, which takes place on **15 or 16 December 2026** (20 minutes per group). Be prepared to explain and justify your implementation choices, results, and conclusions.

Model problem and discretization

Compare sparse direct and iterative methods for the two-dimensional discrete Laplacian: symmetric successive overrelaxation (SSOR), conjugate gradient (CG), and preconditioned conjugate gradient (PCG) with either SSOR or one-level overlapping additive Schwarz (AS) preconditioning.

Consider the Poisson problem on the unit square $\Omega = (0,1)^2$, with unknown solution $u(\boldsymbol{x})$, source function $f(\boldsymbol{x})$, and nonhomogeneous Dirichlet data $b(\boldsymbol{x})$. Here $\boldsymbol{x} = (x, y)$ is the spatial coordinate, $\partial\Omega$ is the boundary, and Δ is the Laplace operator (grad and div denote gradient and divergence):

$$-\Delta u = -\operatorname{div}\operatorname{grad} u = f \qquad \text{in } \Omega, \tag{1}$$

$$u = b \qquad \text{on } \partial\Omega. \tag{2}$$

Divide each coordinate direction into N equal subintervals, with mesh width $h = 1/N$. Discretize (1)–(2) using the second-order central finite-difference approximation with the standard five-point stencil. As in the lecture notes, retain the boundary values as algebraic unknowns and incorporate the boundary conditions so that the matrix remains symmetric positive definite (SPD). The resulting linear system is

$$\boldsymbol{A}^h \boldsymbol{u}^h = \boldsymbol{f}^h.$$

Here $\boldsymbol{A}^h$ is the finite-difference matrix, $\boldsymbol{u}^h$ is the vector of nodal unknowns, and $\boldsymbol{f}^h$ is the right-hand side, including boundary contributions. The superscript h identifies the mesh. Let $\boldsymbol{u}^h_{\text{ex}}$ contain the nodal values of the exact continuous solution u_{ex}. For complexity estimates and experiments, n denotes the total number of unknowns.

For a vector $\boldsymbol{v} = (v_1, \ldots, v_d)^T$ of length d, the Euclidean and maximum norms are $\|\boldsymbol{v}\|_2 = (\sum_{j=1}^{d} |v_j|^2)^{1/2}$ and $\|\boldsymbol{v}\|_\infty = \max_{1 \le j \le d} |v_j|$, respectively. The superscript T denotes transpose.

For every iterative method, use the zero initial guess $\boldsymbol{u}^{h,0} = \mathbf{0}$. Let $\boldsymbol{u}^{h,k}$ denote the approximation after k iterations. Define its residual $\boldsymbol{r}^k$ and the stopping criterion by

$$\boldsymbol{r}^k = \boldsymbol{f}^h - \boldsymbol{A}^h \boldsymbol{u}^{h,k}, \qquad \frac{\|\boldsymbol{r}^k\|_2}{\|\boldsymbol{f}^h\|_2} \leq 10^{-10}. \tag{3}$$

Overlapping domain decomposition

For AS preconditioning, divide the unit square into p nonoverlapping vertical strips of equal width, where p is a positive integer dividing N. A nonnegative integer ℓ controls the extension to overlapping subdomains $\Omega_1, \ldots, \Omega_p$. The quantity $\delta = \ell h$ denotes the one-sided extension distance, not the total intersection width.

Assign every grid unknown, including physical-boundary unknowns, to exactly one strip. State your ownership rule for internal interface points. Enlarge the index set of each strip by ℓ grid columns on each available side, stopping at the physical boundary. Thus $\ell = 0$ gives disjoint index sets, and every unknown is covered for every ℓ; see Figure 1.

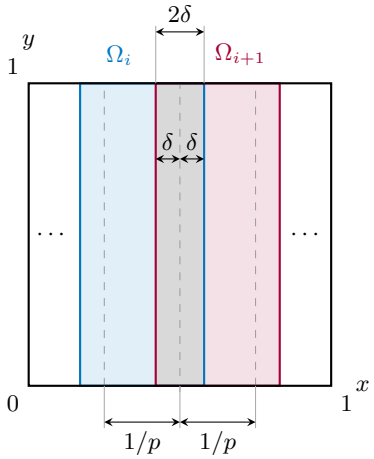


Figure 1: Two neighboring subdomains (schematic). Colored outlines delimit Ω_i and Ω_{i+1}; dashed lines delimit the initial nonoverlapping strips of width $1/p$. The gray intersection straddles their shared interface: each subdomain extends $\delta = \ell h$ into the neighboring strip, with $h = 1/N$. Its geometric width is 2δ before boundary clipping. Discrete grid-point ownership is specified above.

For $i = 1, \ldots, p$, let $\boldsymbol{R}_i$ restrict the global vector of unknowns to those associated with Ω_i. Define the local matrices $\boldsymbol{A}_i$ and the one-level additive Schwarz preconditioning matrix $\boldsymbol{M}_{\mathrm{AS}}$ through

$$\begin{aligned} \boldsymbol{A}_i &= \boldsymbol{R}_i \boldsymbol{A}^h \boldsymbol{R}_i^T, \\ \boldsymbol{M}_{\mathrm{AS}}^{-1} &= \sum_{i=1}^{p} \boldsymbol{R}_i^T \boldsymbol{A}_i^{-1} \boldsymbol{R}_i. \end{aligned} \tag{4}$$

Interpret $\boldsymbol{A}_i^{-1}$ as a local linear solve with an appropriate sparse direct solver.

Pen-and-paper questions (40 points)

1. **Exact solution.** Determine the source function $f(\boldsymbol{x})$ and boundary data $b(\boldsymbol{x})$ such that

   $$u_{\mathrm{ex}}(\boldsymbol{x}) = \sin(xy) \tag{5}$$

   is the exact solution of (1)–(2).

2. **Matrix structure.** Give the size, sparsity structure, and upper and lower bandwidths of $\boldsymbol{A}^h$. Clearly state the ordering of the grid unknowns.

3. **SSOR iteration.** Describe SSOR as a stationary iterative method. Use the lecture convention

   $$\boldsymbol{A}^h = \boldsymbol{D} - \boldsymbol{E} - \boldsymbol{F},$$

   where $\boldsymbol{D}$ contains the diagonal entries, and $-\boldsymbol{E}$ and $-\boldsymbol{F}$ are the strictly lower and upper triangular parts, respectively. Denote the relaxation parameter by ω. Give one complete SSOR iteration and explain why the method can be applied to this linear system.

4. **Computational cost.** Derive the computational cost of one complete SSOR iteration in terms of n, using the sparsity structure of $\boldsymbol{A}^h$. If K iterations are needed to satisfy (3), estimate the total computational cost until convergence. Express your estimates in $\mathcal{O}$-notation; exact operation counts are not required.

5. **Conjugate gradient.**
   (a) Give the CG algorithm and state its requirements on $\boldsymbol{A}^h$.
   (b) Explain how SSOR can be used as a preconditioner for CG, and give the SSOR preconditioner in factored form.
   (c) Compare the work in one SSOR iteration with that in one SSOR-preconditioned CG iteration.

6. **Additive Schwarz.** Consider the one-level additive Schwarz preconditioner (4).
   (a) Explain the roles of the restriction operator $\boldsymbol{R}_i$ and prolongation operator $\boldsymbol{R}_i^T$.
   (b) Explain why $\boldsymbol{A}_i = \boldsymbol{R}_i \boldsymbol{A}^h \boldsymbol{R}_i^T$ represents the discrete operator restricted to Ω_i.
   (c) Explain qualitatively how increasing the overlap affects the local problem sizes and the expected quality of the preconditioner.
   (d) Explain why the local Schwarz solves can be performed independently. Compare this with SSOR preconditioning, which requires a forward and a backward triangular solve. What does this imply for the potential parallelism of the two preconditioners?

Implementation and numerical analysis (60 points)

Use the exact solution in (5) for all numerical experiments. For every experiment, state the mesh size and any subdomain or overlap parameters used. Use the residual in (3) for both stopping tests and convergence plots, including in PCG; do not substitute a preconditioned residual. Count one forward–backward SSOR sweep as one complete iteration.

For timing comparisons, measure the total time spent iterating until (3) is satisfied, excluding matrix assembly and preconditioner setup. Use the same timing routine for all methods and state which routine you used. The Schwarz experiments below also require setup timings. No parallel implementation is required; the pen-and-paper question concerns potential parallelism.

1. **System assembly.** Implement the construction of $\boldsymbol{A}^h$ and $\boldsymbol{f}^h$ using a sparse matrix format.

   Coding assistance: First construct the matrix for the one-dimensional finite-difference operator, then use tensor products to construct the two-dimensional operator. Impose the two-dimensional Dirichlet conditions afterwards, retaining the boundary unknowns and treating the matrix and right-hand side consistently to preserve symmetry.

2. **Discretization error.** Solve the linear system with a sparse direct solver for $h = 2^{-q}$, where q is an integer refinement level. Use several values of q. Using the nodal values of u_{ex} from (5), compute

$$\|\boldsymbol{u}^h - \boldsymbol{u}^h_{\mathrm{ex}}\|_\infty.$$

   Plot this error against h on logarithmic axes. Determine the observed order of convergence and verify second-order accuracy.

3. **SSOR convergence history.** Implement SSOR with $\omega = 1.5$. For several values of n, plot the relative residual $\|\boldsymbol{r}^k\|_2/\|\boldsymbol{f}^h\|_2$ against iteration number k, using a logarithmic scale for the relative residual.

4. **Asymptotic convergence.** For the same SSOR experiments, tabulate the residual reduction factors

$$\frac{\|\boldsymbol{r}^k\|_2}{\|\boldsymbol{r}^{k-1}\|_2}$$

   during the final five complete SSOR iterations. Use these ratios to estimate the asymptotic convergence factor, and discuss how it changes under mesh refinement.

5. **SSOR computational cost.** Measure the central processing unit (CPU) time needed to satisfy (3). For each problem size, report n, the number of iterations, and CPU time. Compare the measurements with the complexity estimate derived in the pen-and-paper questions.

6. **CG and SSOR-preconditioned CG.** Solve the system using CG without preconditioning and SSOR-preconditioned CG, using $\omega = 1.5$ for the SSOR preconditioner. Use the same problem sizes as for SSOR. For each method, plot $\|\boldsymbol{r}^k\|_2/\|\boldsymbol{f}^h\|_2$ against k, using a logarithmic scale for the relative residual. For each problem size, report n, the number of iterations, and CPU time.

   Compare SSOR, CG, and SSOR-preconditioned CG in terms of convergence, CPU time, and mesh refinement, and relate the measurements to the complexity estimate derived in the pen-and-paper questions.

7. **Additive Schwarz implementation.** Implement the one-level additive Schwarz preconditioner (4) and use it in PCG. Use the subdomain index sets defined above. In a single setup phase, construct

   - the subdomain index sets;
   - the local matrices $\boldsymbol{A}_i$;
   - the sparse factorizations for the local solves.

   Reuse these setup objects throughout PCG. Do not form $\boldsymbol{A}_i^{-1}$ explicitly. For a global residual $\boldsymbol{r}$, let $\boldsymbol{r}_i$ be its restriction to Ω_i, $\boldsymbol{z}_i$ the local correction, and $\boldsymbol{z}$ the resulting preconditioned residual. Compute these by restricting, solving locally, and accumulating:

$$\begin{aligned} \boldsymbol{r}_i &= \boldsymbol{R}_i \boldsymbol{r}, & i &= 1, \ldots, p, \\ \boldsymbol{A}_i \boldsymbol{z}_i &= \boldsymbol{r}_i, & i &= 1, \ldots, p, \\ \boldsymbol{z} &= \sum_{i=1}^{p} \boldsymbol{R}_i^T \boldsymbol{z}_i. \end{aligned}$$

   Verify that additive Schwarz-preconditioned CG converges to the same discrete solution as the sparse direct solver.

8. **Effect of subdomains and overlap.** Choose one sufficiently fine mesh and keep it fixed while varying p. Use divisors of N for p so that the nonoverlapping strips align with the grid. For each p, vary the number of overlapping grid layers ℓ. Include at least $\ell = 0, 1, 2, 4$ whenever meaningful for the chosen decomposition.

   For every experiment, report p, ℓ, $\delta = \ell h$, the number of PCG iterations needed to satisfy (3), and the setup time and total PCG time to convergence (excluding setup). Include construction and factorization of the local problems in the setup time.

   Discuss the following questions:

   (a) How does increasing the overlap affect the number of iterations?

   (b) How does increasing p affect convergence for fixed h and fixed overlap?

   (c) How does the increased cost of the local problems compete with the improved convergence from larger overlap?

   (d) Compare CG, SSOR-preconditioned CG, and additive Schwarz-preconditioned CG for one fixed problem size.